GENOMIC SEQUENCE CLASSIFICATION: COMPLETE END-TO-END ARCHITECTURE

Pangenome-based pathogenicity research + APP / PSEN1 / PSEN2 gene classification | Implementation-based system view

1. INPUTS & ENTRY POINTS
Data, labels and user access

2. DATA PREPROCESSING
Graph / sequence preparation

3. INPUT REPRESENTATION
Tokens / engineered features

4. TRAINING & SELECTION
Fit only on training partitions

5. INFERENCE & PREDICTION
Reuse trained model state

6. EVALUATION & OUTPUTS
Held-out results and artifacts

A | PANGENOME GRAPH -> DNABERT

Command-line workflow: main.py | BENIGN / PATHOGENIC

Pangenome graph input
GFA / rGFA subset
Segments, links, named paths
sample.gfa or a supplied graph

Optional graph preparation
ClinVar SNVs or fallback records
Allele bubbles + synthetic flanks
download_dataset.py: demonstration helper

Annotations + pretrained assets
CSV / TSV / VCF class labels
Local or Hugging Face checkpoint
Path-to-label correspondence must be verified

Command-line configuration
Graph, k, length, path caps, seed
--train / --model / --labels
main.py + utils.PipelineConfig

Parse GFA records
Read H / S / L / P records
Validate segment DNA
graph_processing.py

Construct directed graph
Segments -> sequence nodes
Links -> directed edges
NetworkX DiGraph

Contract cycles into a DAG
Strongly connected components
Sorted member concatenation
SCC map retains member identifiers

Extract paths and linearize
Capped DFS + seeded random walks
Deduplicate paths; concatenate DNA
path_extraction.py + sequence_processing.py

Overlapping k-mer tokens
Sliding window, stride = 1
k = 3 / 4 / 5 / 6
tokenizer_module.KmerTokenizer

Checkpoint-aligned token IDs
Load checkpoint tokenizer vocabulary
Reject incompatible k-mer vocabulary
Token IDs must match embedding rows

Fixed-length input encoding
[CLS] + k-mers + [SEP] + [PAD]
Default L = 128; warn on truncation
sequence_processing.encode_tokens

BERT-ready model inputs
input_ids + attention_mask: [B, L]
BERT generates positions internally
Original DNA retained in EncodedSequence

Label mapping and grouped split
Identical encoded inputs stay together
~56% train / 14% val / 30% test
Reject conflicting groups / insufficient classes

Fine-tune DNABERT + binary head
Class-weighted cross-entropy; AdamW
Train partition only; CPU / CUDA
dnabert_model.py | fixed epoch count

Validation threshold selection
Maximize validation binary F1
0.05-0.95 grid; ties prefer 0.5
Never choose the threshold on test labels

Persist model and experiment
Weights + config + tokenizer
Threshold + run metadata
output_dir / experiment.json

New graph or held-out input
Apply the same preprocessing
Same checkpoint k and length
Reuse stages 2-3 for inference inputs

Load a trained classifier
Local checkpoint or trained model
Reject an untrained classifier head
No random-model fallback

Transformer forward pass
BERT encoder -> classification logits
Softmax -> two class probabilities
No gradient updates during prediction

Apply fixed class threshold
p(PATHOGENIC) >= threshold
BENIGN (0) / PATHOGENIC (1)
Saved threshold; otherwise default 0.5

Evaluate held-out test inputs
Training run: untouched test partition
Inference: metrics only with labels
Checkpoint / label independence still matters

Classification metrics
Accuracy, precision, recall, binary F1
ROC-AUC, average precision, matrix
No accuracy claim without verified labels

Durable experiment record
Splits, counts, config, training history
Package versions + input SHA-256
experiment.json includes test metrics

Research prediction output
Console class + probabilities
Saved checkpoint reused in later runs
Not a verified clinical interpretation

A and B are two execution paths within one project.

Reserved holdout -> selected pipeline -> test evaluation

B | FASTA GENE SEQUENCES -> CLASSICAL MACHINE LEARNING

Browser + CLI workflow | Healthy / Unhealthy

Gene-organized FASTA dataset
APP / PSEN1 / PSEN2 sequences
Healthy and Unhealthy subfolders
Dataset folder selected by the user

Folder-derived class labels
output_safe_healthy -> 0
output_safe_unhealthy -> 1
Label provenance needs independent review

Browser or command line
Flask dashboard: GET /
gene_classifier.py experiment CLI
templates/index.html + app.py

New prediction upload
FASTA records + selected gene
POST /predict after web training
Enters inference at stage 5, not training

Load and clean FASTA files
Uppercase; retain A / C / G / T
Collect sequence, label, gene, file
gene_classifier.load_data / read_fasta

Audit dataset integrity
Exact duplicates + label conflicts
Report train / holdout overlap
POST /audit or --audit-only; does not block

Reserve a stratified holdout
Stratify by gene + class; seed 42
Default 70% training / 30% test
validation_pipeline.run_experiment

Build training-only CV folds
5-fold stratification by gene + class
Identical folds for every candidate
At least 5 training samples per gene/class

Learn fold-specific references
First healthy reference per gene
Fit on each training fold only
GeneFeatures.fit

Extract sequence feature families
Global k-mers + 6 composition stats
11 mutation + context + gene IDs
All features at k = 4: 532 dimensions

Fold-contained transformations
Optional mutual-information selection
Fit StandardScaler on training only
Reuse fitted transforms for validation/test

Training-only imbalance handling
Optional SMOTE within classifier.fit
Class weighting in model fitting
No validation / test resampling

Train candidate classifiers
Random Forest / SVM / LR / GB
XGBoost / LightGBM if installed
Optional tuning + soft voting via CLI/engine

Select with cross-validation
Compare models / feature subsets
Highest mean CV macro F1 wins
Never choose the winner using holdout scores

Refit the winning pipeline
Full training partition only
Features -> scale -> classifier
Selected preprocessing travels with the model

Retain active web model
Fitted pipeline in Flask _state
Replaced by retraining; lost on restart
app.py does not auto-load model.pkl

Validate uploaded FASTA
1-100 records; <=16 MB request
ACGT only; reference length +/-5%
Selected gene must have a trained reference

Reuse the fitted pipeline
Same references, features and scaler
Do not refit on uploaded sequences
Process-local Flask pipeline required

Classify each uploaded record
pipeline.predict + predict_proba
Healthy (0) / Unhealthy (1)
No automatic gene / alignment verification

Return browser prediction JSON
Record header, gene, length, class
Probability, counts, research warning
POST /predict -> dashboard

Evaluate the reserved holdout
Only after candidate selection/refit
Per-gene test results also reported
Exact duplicates are audited, not grouped

Metrics and model comparison
Accuracy; macro / weighted F1
Per-class precision/recall; AUC / AP
CV means / std + confusion matrix

Visualize experiment results
ROC + PR curves; confusion matrix
Feature importance where supported
Dashboard JSON; CLI can save plot PNGs

Present research findings
Dataset summary, audit, split counts
Comparison table + prediction results
Model probabilities are not disease risks

READING GUIDE: solid arrows = processing / model flow; dashed route = classical holdout evaluation. Uploads enter stage 5 directly.

USER INTERFACE & ORCHESTRATION
Flask on localhost:5000: /, /audit, /train, /predict.
Browser: Bootstrap + Chart.js; external CDN assets.
Training requests are synchronous; no job queue.
DNABERT runs separately through main.py CLI.

RUNTIME, STORAGE & REPRODUCIBILITY
Local files + in-memory Python / NetworkX objects.
PyTorch: CPU or available CUDA; scikit-learn: local.
DNABERT: persistent checkpoint / tokenizer / JSON.
No database, distributed worker or model registry.

SCIENTIFIC BOUNDARIES
SCC order, strand orientation and overlaps are simplified.
Long DNABERT inputs truncate; labels need path mapping.
Helper-generated graph flanks are synthetic, not loci.
Related-sample leakage and clinical validity not resolved.

SUPPORTING CODE & QUALITY CHECKS
utils.py: configuration, seed usage and logging support.
Tests: test_dnabert_pipeline + test_validation_pipeline.
generate_report.py: static report helper, not live results.
No automatic fusion, diagnosis or proven accuracy gain.